⚡ HackForge 2026 - Master IoT & Dashboard Field Guide

Complete single-page reference for connecting Arduino hardware with Web Dashboard (Node.js & Spring Boot).

⏱️ 6-Hour Hackathon | 🤖 3× 15-min AI Lifelines | 🔌 Arduino Uno + WebSockets | 🌍 15% SDG Weightage

📐 1. System Architecture: How Hardware Talks to Software

The system operates in 3 simple stages passing real-time sensor data from physical pins to the browser:

1. Arduino Uno (Hardware)

Reads pins (e.g. A0 = 28.5°C), converts voltage to engineering units, and prints a delimited key-value line to the USB cable:

```
TEMP:28.5,GAS:420,LIGHT:800\n
```

2. Backend Bridge (Node / Spring)

Runs locally on laptop. Listens to USB COM port (COM3), splits readings by commas, checks alert thresholds, and broadcasts JSON over WebSockets:

```
{"TEMP": 28.5, "GAS": 420}
```

3. Web Dashboard (Browser)

Opens on `localhost`. Receives live WebSocket stream in sub-millisecond time and automatically updates animated metric cards, live line charts, and event logs.

💡 **100% Offline & Reliable:** No WiFi modules (ESP8266/ESP32) or cloud servers needed. Everything communicates offline through the USB cable!

📦 2. Adapting Code to Mystery Box Components (4 Steps)

1 Identify your Mystery Sensors

Inspect the components in your box (e.g. Temperature, Gas/Smoke, Light LDR, Ultrasonic Distance, Soil Moisture, Relay, Buzzer, LED).

2 Edit the Single Configuration File

You only need to edit **ONE** file in your project to update all titles, labels, units, ranges, and alert thresholds:

If using Spring Boot (DashboardConfig.java):

```
public static final List<SensorInfo> SENSORS = List.of(
  new SensorInfo("TEMP", "Temperature", "°C", 0, 100, 60, "above", "🌡️", "#ff6b
  new SensorInfo("GAS",  "Air Quality",  "PPM", 0, 1000, 400, "above", "💨", "#
  new SensorInfo("DIST", "Water Level", "cm", 0, 200, 30,  "below", "📏", "#38b
);
```

If using Node.js (server.js):

```
const SENSOR_CONFIG = [
  { key: "TEMP", label: "Temperature", unit: "°C", min: 0, max: 100, alertThresh
  { key: "GAS",  label: "Air Quality",  unit: "PPM", min: 0, max: 1000, alertThr
];
```

3 Format Arduino Serial Output

In your Arduino sketch, print matching `KEY:VALUE` pairs separated by commas and ending with a newline:

```
Serial.print("TEMP:"); Serial.print(temp);
Serial.print(",GAS:"); Serial.print(gasVal);
Serial.print(",DIST:"); Serial.println(distance); // Note: println ends with '\n'
```

4 Switch to Real Arduino Mode

In `DashboardConfig.java` or `server.js`, set `USE_MOCK_DATA = false` and set your COM port (e.g. `"COM3"`).

3. Sensor Wiring & C++ Formula Cheat Sheet

Copy-paste these formulas into your Arduino `loop()` function:

SENSOR	WIRING (ARDUINO UNO)	ARDUINO C++ READING CODE
LM35 Temperature	VCC(5V), GND, OUT → A0	`float temp = (analogRead(A0) * 5.0 / 1024.0) * 100.0;`
MQ-2 / MQ-135 Gas	VCC(5V), GND, AOUT → A1	`int gasPpm = analogRead(A1);`
LDR Light Sensor	5V, 10kΩ resistor, OUT → A2	`int light = analogRead(A2);`

HC-SR04 Ultrasonic	VCC(5V), GND, Trig→9, Echo→10	`digitalWrite(9, LOW); delayMicroseconds(2);`
		`digitalWrite(9, HIGH); delayMicroseconds(10);`
		`digitalWrite(9, LOW);`
		`long dur = pulseIn(10, HIGH);`
		`float dist = dur * 0.034 / 2;`

Soil Moisture / Potentiometer	VCC(5V), GND, SIG → A3	`int moist = map(analogRead(A3), 1023, 0, 0, 100);`
PIR Motion Sensor	VCC(5V), GND, OUT → Pin 2	`int motion = digitalRead(2); // 1=Motion, 0=None`

🚀 4. How to Run Both Dashboards

Option A: Node.js (Fastest)

```
# 1-Click: Double click RUN_NODE_
# Or via terminal:
cd c:\uves\Antigravity\Hackforge\
npm start
```

Dashboard URL: `http://localhost:3000`

Option B: Spring Boot (Java JDK 25)

```
# In IntelliJ IDEA / Eclipse:
Open IotDashboardApplication.java
# Or via terminal:
mvn spring-boot:run
```

Dashboard URL: `http://localhost:8080`

🌍 5. SDG Alignment Matrix (15% Weightage - Section 8)

Map whatever mystery sensors you receive to an SDG goal to secure the 15 marks:

SDG 3: Good Health

Sensors: Gas + Temperature + Buzzer
Project: Smart ICU / Indoor Air Quality & Hazard Monitor.

SDG 6: Clean Water

Sensors: Ultrasonic + Soil/Water + Relay
Project: Automated Flood Level Warning & Reservoir Management.

SDG 7: Clean Energy

Sensors: LDR (Light) + PIR Motion + Relay
Project: Smart Adaptive Street Lighting & Energy Conservation.

SDG 11: Sustainable Cities

Sensors: Gas + Flame + Ultrasonic
Project: Smart City Disaster & Waste Management System.

SDG 12: Responsible Consumption

Sensors: Soil Moisture + Temp + Relay (Pump)
Project: Precision Agriculture Smart Irrigation System.

SDG 13: Climate Action

Sensors: Temperature + Gas + LDR + Fan
Project: Urban Heat Island & Emission Monitoring Dashboard.

🎤 6. Judges' Technical Question Round (Rules 7.3 & 7.4)

Memorize these straightforward answers to ace the technical questions:

❓ "How does the hardware communicate with the software dashboard?"

> "The Arduino reads analog and digital sensor voltages, converts them into engineering units (°C, PPM, lux), and transmits a formatted delimited string (`KEY:VALUE`) over USB Serial at 9600 baud. The backend listens on the COM port, parses the values in real-time, and broadcasts JSON packets to the browser over persistent WebSockets."

❓ "Why did you use WebSockets instead of regular HTTP REST polling?"

> "HTTP polling creates unnecessary latency and network overhead by constantly sending requests every few seconds. WebSockets provide a full-duplex, persistent TCP channel. When an emergency spike or alert threshold is crossed, the server pushes the alert to the dashboard in sub-millisecond latency."

❓ "How does your system handle safety thresholds and actuation?"

> "We implement two-tier safety: Tier 1 is autonomous on the Arduino, immediately triggering buzzers/relays if safety limits are exceeded. Tier 2 is software-level on our web dashboard, showing real-time warning banners, logging timestamped events, and providing manual override buttons."

🛠️ 7. Troubleshooting & Emergency Fixes

1. **Error: "Access is denied" / "Port Busy" on COM3**

   Cause: Arduino IDE's Serial Monitor is open, locking the COM port.

   Fix: Close the Serial Monitor window in Arduino IDE before starting Node.js or Spring Boot!

2. **Garbled characters (e.g. ؟؟?x#) on Serial**

   Cause: Baud rate mismatch.

   Fix: Ensure Arduino `Serial.begin(9600)` and backend `BAUD_RATE = 9600` match exactly.

3. **Sensor readings are fluctuating wildly**

   Cause: Loose GND jumper wire or ungrounded breadboard rail.

   Fix: Ensure all sensors share a common GND rail connected to Arduino's GND pin.